

Travaux pratiques - Contrôle des versions logicielles avec Git

Objectifs

Partie 1 : Lancer la DEVASC VM

Partie 2 : Initialisation de Git

Partie 3 : Mise en scène et validation d'un fichier dans le référentiel Git

Partie 4 : Gestion du fichier et suivi des modifications

Partie 5 : Succursales et fusion

Partie 6 : Gestion des conflits de fusion

Partie 7 : Intégration de Git avec GitHub

Contexte/scénario

Dans ce TP, vous explorerez les principes fondamentaux du système de contrôle de version distribué Git, y compris la plupart des fonctionnalités que vous devez connaître pour collaborer à un projet logiciel. Vous allez également intégrer votre dépôt Git local avec le dépôt GitHub basé sur le cloud.

Ressources requises

- 1 PC avec système d'exploitation de votre choix
- Boîte virtuelle ou VMWare
- Machine virtuelle DEVASC

Instructions

Partie 1 : Lancer la machine virtuelle DEVASC

Si vous n'avez pas encore terminé **TP - Installer l'environnement de laboratoire de machines virtuelles**, faites-le maintenant. Si vous avez déjà terminé ce TP, lancez la machine virtuelle DEVASC maintenant.

Partie 2 : Initialisation en cours

Dans cette partie, vous allez initialiser un dépôt Git.

Étape 1: Ouvrez un terminal dans le DEVASC-LABVM.

Double-cliquez sur l'icône de l'émulateur de terminal sur le bureau pour ouvrir une fenêtre de terminal.

Étape 2: Initialiser un référentiel Git.

- Utilisez la commande **ls** pour afficher une liste du répertoire en cours. Rappelez-vous que les commandes sont sensibles à la casse.
- Ensuite, configurez les informations utilisateur à utiliser pour ce référentiel local. Cela associera vos informations au travail que vous contribuez à un référentiel local. Utilisez votre nom à la place de « Exemple d'utilisateur » pour le nom entre guillemets " ». Utilisez @example .com pour votre adresse e-mail.

Remarque : ces paramètres peuvent être tout ce que vous voulez à ce stade. Toutefois, lorsque vous réinitialisez ces valeurs globales dans la partie 7, vous utiliserez le nom d'utilisateur pour votre compte GitHub. Si vous le souhaitez, vous pouvez utiliser votre nom d'utilisateur GitHub maintenant.

- À tout moment, vous pouvez consulter ces paramètres à l'aide de la commande **git config —list**.

- d. Utilisez la commande **cd** pour accéder au dossier **devnet-src** :
- e. Faites un répertoire **git-intro** et changez le répertoire dedans :
- f. Utilisez la commande **git init** pour initialiser le répertoire courant (git-intro) en tant que dépôt Git. Le message affiché indique que vous avez créé un référentiel local dans votre projet contenu dans le répertoire masqué **.git**. C'est là que se trouve l'historique de vos modifications. Vous pouvez le voir avec la commande **ls -a**.
- g. Lorsque vous travaillez sur votre projet, vous voudrez vérifier quels fichiers ont changé. Ceci est utile lorsque vous validez des fichiers dans le repo, et que vous ne voulez pas les valider tous. La commande **git status** affiche les fichiers modifiés dans le répertoire de travail qui sont mis en scène pour votre prochain commit.

Ce message vous indique :

- Que vous êtes sur le maître de branche. (Les branches sont discutées plus loin dans ce laboratoire)
- Le message de validation est la validation initiale.
- Il n'y a rien de changé à commettre.

Vous verrez que le statut de votre repo changera une fois que vous aurez ajouté des fichiers et que vous commencerez à apporter des modifications.

Partie 3 : Mise en place et validation d'un fichier dans le référentiel

Dans cette partie, vous allez créer un fichier, mettre en scène ce fichier et valider ce fichier dans le dépôt Git.

Étape 1: Créez un fichier journal.

- a. Le dépôt **git-intro** est créé mais vide. À l'aide de la commande **echo**, créez le fichier **DEVASC.txt** avec les informations contenues entre guillemets.

```
"I am on my way to passing the Cisco DEVASC exam"
```
- b. Utilisez la commande **ls -la** pour vérifier le fichier, ainsi que le répertoire **.git**, se trouvent dans le répertoire **git intro**. Ensuite, utilisez **cat** pour afficher le contenu de **DEVASC.txt**.

Étape 2: Examinez l'état du référentiel.

Examinez l'état du référentiel en utilisant **git status**. Notez que Git a trouvé le nouveau fichier dans le répertoire, et sait qu'il n'est pas suivi.

Étape 3: Mise en scène du fichier.

- a. Ensuite, utilisez la commande **git add** pour « mettre en scène » le fichier **DEVASC.txt**. La mise en scène est une phase intermédiaire avant la validation d'un fichier dans le référentiel avec la commande **git commit**. Cette commande crée un instantané du contenu du fichier au moment de la saisie de cette commande. Toute modification du fichier nécessite une autre commande **git add** avant de valider le fichier.
- b. En utilisant à nouveau la commande **git status**, notez les modifications mises en page affichées sous la forme « new file : DEVASC.txt ».

Étape 4: Validation d'un fichier.

Maintenant que vous avez mis en scène vos modifications, vous devrez les valider afin de laisser Git savoir que vous voulez commencer à suivre ces modifications. Validez votre contenu en tant que nouvel instantané de validation à l'aide de la commande **git commit**. Le commutateur de **message-m** vous permet d'ajouter un message expliquant les modifications que vous avez apportées. Notez la combinaison de chiffres et de lettres en surbrillance dans la sortie. Ceci est l'ID de validation. Chaque commit est identifié par un hachage SHA1

unique. L'ID de validation est les 7 premiers caractères du hachage de validation complet. Votre ID de validation sera différent de celui affiché.

```
"Committing DEVASC.txt to begin tracking changes"
```

Étape 5: Affichage de l'historique des validations.

Utilisez la commande **git log** pour afficher tous les validations dans l'historique de la branche courante. Par défaut, toutes les validations sont effectuées dans la branche maître. (Les branches seront discutées plus tard.) La première ligne est le hachage de validation avec l'ID de validation comme 7 premiers caractères. Le fichier est validé dans la branche maître. Ceci est suivi de votre nom et adresse e-mail, de la date du commit et du message que vous avez inclus avec le commit.

Partie 4 : Modification du fichier et suivi des modifications

Dans cette partie, vous allez modifier un fichier, mettre en scène le fichier, valider le fichier et vérifier les modifications dans le référentiel.

Étape 1: Modifiez le fichier enregistré.

- Effectuez une modification à DEVASC.txt à l'aide de la commande **echo**. Assurez-vous d'utiliser ">>" pour ajouter le fichier existant. Le ">" écrasera le fichier existant. Utilisez la commande **cat** pour afficher le fichier modifié.

```
"I am beginning to understand Git!"
```

- Utilisez la commande **cat** pour afficher le fichier modifié.

Étape 2: Vérifiez la modification apportée au référentiel.

Vérifiez la modification dans le référentiel à l'aide de la commande **git status**.

Étape 3: Déposez le fichier modifié.

Le fichier modifié devra être remis en scène avant de pouvoir être validé à l'aide de la commande **git add**.

Étape 4: Valider le fichier de la mise en scène.

Valider le fichier intermédiaire à l'aide de la commande **git commit**. Remarquez le nouvel ID de validation.

Étape 5: Vérifiez les modifications apportées au référentiel.

- Utilisez à nouveau la commande **git log** pour afficher toutes les validations. Notez que le journal contient l'entrée de validation d'origine ainsi que l'entrée de la validation que vous venez d'effectuer. Le dernier commit est affiché en premier. La sortie met en évidence l'ID de validation (7 premiers caractères du hachage SHA1), la date/heure de la validation et le message de la validation pour chaque entrée.
- Lorsque vous avez plusieurs entrées dans le journal, vous pouvez comparer les deux validations en utilisant la commande **git diff** ajoutant d'abord l'ID de commit original et le dernier commit second : **git diff <commit ID original> <commit ID latest>**. Vous devrez utiliser vos identifiants de validation. Le signe "+" à la fin, suivi du texte, indique le contenu qui a été ajouté au fichier.

```
git diff b510f8e 9f5c4c5
```

Partie 5 : Succursales et fusion

Lorsqu'un référentiel est créé, les fichiers sont automatiquement placés dans une branche appelée **master**. Dans la mesure du possible, il est recommandé d'utiliser des branches plutôt que de mettre directement à jour la branche maître. La ramification (branchement) est utilisée pour que vous puissiez apporter des modifications dans une autre zone sans affecter la branche principale. Ceci est fait pour aider à prévenir les mises à jour accidentelles qui pourraient remplacer le code existant.

Dans cette partie, vous allez créer une nouvelle branche, extraire la branche, apporter des modifications dans la branche, mettre en scène et valider la branche, fusionner les modifications de branche dans la branche maître, puis supprimer la branche.

Étape 1: Créer une nouvelle branche

Créez une nouvelle branche appelée **fonctionnalité** à l'aide de la `<branch-name>` commande **git branch**.

Étape 2: Vérifier la branche actuelle

Utilisez la commande **git branch** sans nom de branche pour afficher toutes les branches de ce référentiel. Le `"**"` à côté de la branche principale indique qu'il s'agit de la branche actuelle — la branche actuellement « récupérée ».

Étape 3: Commander la nouvelle succursale

Utilisez la `<branch-name>` commande **git checkout** pour basculer vers la branche de *feature*.

Étape 4: Vérifier la branche actuelle

- Vérifiez que vous avez basculé vers la branche *feature* à l'aide de la commande **git branch**. Notez le `"**"` à côté de la branche de *feature*. C'est maintenant la *branche de travail*.
- Ajoutez une nouvelle ligne de texte au fichier DEVASC.txt, à nouveau en utilisant la commande **echo** avec les signes `">>"`.

```
"This text was added originally while in the feature branch"
```
- Vérifiez que la ligne a été ajoutée au fichier à l'aide de la commande **cat**.

Étape 5: Déposez le fichier modifié dans la branche de fonctionnalité.

- Déposez le fichier mis à jour vers la branche *d'entité* active.
- Utilisez la commande **git status** et notez que le fichier modifié **DEVASC.txt** est mis en scène dans la branche de *fonctionnalité*.

Étape 6: Valider le fichier stagé dans la branche de fonctionnalité.

- Valider le fichier intermédiaire à l'aide de la commande **git commit**. Notez le nouvel ID de validation et votre message.

```
"Added a third line in feature branch"
```
- Utilisez la commande **git log** pour afficher toutes les validations, y compris le commit que vous venez de faire à la branche de *fonctionnalité*. La validation précédente a été effectuée dans la branche *master*.

Étape 7: Extraire la branche principale.

Passez à la branche *master* à l'aide de la commande **git checkout master** et vérifiez la branche de travail actuelle à l'aide de la commande **git branch**.

Étape 8: Fusionner le contenu du fichier de la fonction vers la branche principale.

- Les branches sont souvent utilisées lors de l'implémentation de nouvelles fonctionnalités ou correctifs. Ils peuvent être soumis pour examen par les membres de l'équipe, puis une fois vérifiés, ils peuvent être tirés dans la base de code principale, la branche maître.

Fusionnez le contenu (connu sous le nom d'historique) de la branche d'entité dans la branche maître à l'aide de la `<branch-name>` commande **git merge**. Le nom de branche est la branche à partir de laquelle les histoires sont extraites dans la branche actuelle. La sortie affiche qu'un fichier a été modifié avec une ligne insérée.

- b. Vérifiez le contenu ajouté au fichier DEVASC.txt dans la branche maître à l'aide de la commande **cat**.

Étape 9: Suppression d'une branche.

- a. Vérifiez que la branche de **fonctionnalité** est toujours disponible à l'aide de la commande **git branch**.
- b. Supprimez la branche de **feature** à l'aide de la <branch-name> commande **git branch -d**.
- c. Vérifiez que la branche de fonctionnalité n'est plus disponible à l'aide de la commande **git branch**.

Partie 6 : Gestion des conflits de fusion

Parfois, vous pouvez rencontrer un conflit de fusion. C'est alors que vous avez peut-être apporté des modifications superposées à un fichier et que Git ne peut pas fusionner automatiquement les modifications.

Dans cette partie, vous allez créer une branche de test, modifier son contenu, mettre en scène et valider la branche test, basculer vers la branche maître, modifier à nouveau le contenu, mettre en scène et valider la branche maître, tenter de fusionner les branches, localiser et résoudre le conflit, mettre en scène et valider la branche maître à nouveau, et vérifier votre commit.

Étape 1: Créez un nouveau test de branche.

Créez un nouveau **test** de branche.

Étape 2: Vérification du test de branche.

- a. Passer à la commande (**checkout**) le **test** de branche.
- b. Vérifiez que la branche de travail est la branche de **test**.

Étape 3: Vérifiez le contenu actuel de DEVASC.txt.

Vérifiez le contenu actuel du fichier **DEVASC.txt**. Notez que la première ligne comprend le mot "Cisco".

Étape 4: Modifiez le contenu de DEVASC.txt dans la branche de test.

Utilisez la commande **sed** pour remplacer le mot "Cisco" par "NetAcad" dans le fichier **DEVASC.txt**.

Étape 5: Vérifiez le contenu du fichier DEVASC.txt modifié dans la branche de test.

Vérifiez la modification apportée au fichier **DEVASC.txt**.

Étape 6: Stage et validation de la branche de test.

Stage et validation du fichier avec une seule commande **git commit -a**. L'option **-a** n'affecte que les fichiers qui ont été modifiés et supprimés. Cela n'affecte pas les nouveaux fichiers.

Étape 7: Checkout the master branch.

- a. Checkout (switch to) the **master** branch.
- b. Vérifiez que la branche maître est votre branche de travail actuelle.

Étape 8: Modifiez le contenu de DEVASC.txt dans la branche maître.

Utilisez la commande **sed** pour changer le mot "Cisco" en "DevNet" dans le fichier DEVASC.txt.

Étape 9: Vérifiez le contenu du fichier DEVASC.txt modifié dans la branche maître.

Vérifiez la modification apportée au fichier.

Étape 10: Stage et validation de la branche maître

Stage et validation du fichier à l'aide de la commande **git commit -a**.

```
"Changed Cisco to DevNet"
```

Étape 11: Essayez de fusionner la branche de test dans la branche maître.

Essayez de fusionner l'historique de la branche de test dans la branche maître.

Étape 12: Trouve le conflit.

- Utilisez la commande **git log** pour afficher les validations. Notez que la version HEAD est la branche principale. Cela sera utile dans la prochaine étape.
- Utilisez la commande **cat** pour afficher le contenu du fichier DEVASC.txt. Le fichier contient maintenant des informations pour vous aider à trouver le conflit. La version HEAD (branche principale) contenant le mot "DevNet" est en conflit avec la version de la branche test et le mot "NetAcad".

Étape 13: Modifiez manuellement le fichier DEVASC.txt pour supprimer le texte en conflit.

- Utilisez la commande **vim** pour modifier le fichier.
- Utilisez les flèches haut et bas pour sélectionner la ligne de texte appropriée. Appuyez sur **dd** (delete) sur les lignes suivantes qui sont mises en surbrillance. **dd** supprimera la ligne sur laquelle se trouve le curseur.
- Enregistrez vos modifications dans vim en appuyant sur **ESC** (la touche d'échappement), puis en tapant : (deux points) suivi de **wq** et en appuyant sur entrée.

Étape 14: Vérifiez vos modifications de DEVASC.txt dans la branche maître.

Vérifiez que vous avez modifié à l'aide de la commande **cat**.

Étape 15: Stage et validation de la branche maître

Stage et validation DEVASC.txt dans la branche maître à l'aide de la commande **git commit -a**.

```
"Manually merged from test branch"
```

Étape 16: Vérifiez la validation.

Utilisez la commande **git log** pour vérifier la validation. Si nécessaire, vous pouvez utiliser **q** pour quitter l'affichage git log.

Partie 7 : Intégration de Git avec GitHub

Jusqu'à présent, toutes les modifications que vous avez apportées à votre fichier ont été stockées sur votre machine locale. Git s'exécute localement et ne nécessite aucun serveur de fichiers central ou service d'hébergement basé sur le cloud. Git permet à un utilisateur de stocker et de gérer localement des fichiers.

Bien que Git soit utile pour un seul utilisateur, l'intégration du référentiel Git local avec un serveur basé sur le cloud comme GitHub est utile lorsque vous travaillez au sein d'une équipe. Chaque membre de l'équipe conserve une copie sur le référentiel sur sa machine locale et met à jour le référentiel central basé sur le cloud pour partager les modifications éventuelles.

Il y a pas mal de services Git populaires, y compris GitHub, Stash d'Atlassian et GitLab. Parce qu'il est facilement accessible, vous utiliserez GitHub dans ces exemples.

Étape 1: Créez un compte GitHub.

Si vous ne l'avez pas déjà fait, rendez-vous sur github.com et créez un compte GitHub. Si vous avez un compte GitHub passez à l'étape 2.

Étape 2: Connectez-vous à votre compte GitHub Créer un référentiel.

Connectez-vous à votre compte GitHub.

Étape 3: Créer un référentiel

- Sélectionnez le bouton **"New repository"** ou cliquez sur l'icône "+" dans le coin supérieur droit et sélectionnez **"New repository"**.
- Créez un référentiel à l'aide des informations suivantes :
Nom du référentiel: **devasc-study-team**
Description: **Travailler ensemble pour réussir l'examen DEVASC**
Public/Private: **Private**
- Sélectionner : **Créer un référentiel**

Étape 4: Créer un nouveau répertoire devasc-study-team.

- Si vous n'êtes pas déjà dans le répertoire **git-intro**, changez-y maintenant.
- Créez un nouveau répertoire appelé devasc-study-team. Le répertoire n'a pas besoin de correspondre au nom en tant que référentiel.

Étape 5: Changer le répertoire en devasc-study-team.

Utilisez la commande **cd** pour changer les répertoires en **devasc-study-team**.

Étape 6: Copiez le fichier DEVASC.

- Utilisez la commande **cp** pour copier le fichier **DEVASC.txt** du répertoire parent **git-intro** vers le sous-répertoire **devasc-study-team**. Les deux points et une barre oblique précédant le nom du fichier indiquent le répertoire parent. L'espace et la période qui suivent le nom du fichier indiquent de copier le fichier dans le répertoire actif avec le même nom de fichier.
- Vérifiez que le fichier a été copié avec la commande **ls** et le contenu du fichier avec la commande **cat**.

Étape 7: Initialisez un nouveau dépôt Git.

- Utilisez la commande **git init** pour initialiser le répertoire courant (devasc-study-team) en tant que dépôt Git. Le message affiché indique que vous avez créé un référentiel local dans votre projet contenu dans le répertoire masqué **.git**. C'est là que se trouve l'historique de vos modifications.
- Ensuite, vérifiez vos variables git globales avec la commande **git config —list**.
- Si les variables **user.name** et **user.email** ne correspondent pas à vos informations d'identification GitHub, modifiez-les maintenant.

```
git config --global user.name "GitHub username"
git config --global user.email GitHub-email-address
```

Étape 8: Dépôt Git Point vers le dépôt GitHub.

- Utilisez la commande **git remote add** pour ajouter une URL Git comme alias distant. La valeur "origin" pointe vers le dépôt nouvellement créé sur GitHub. Utilisez votre nom d'utilisateur GitHub dans le chemin d'accès URL pour **github-username**.

Remarque : Votre nom d'utilisateur est sensible à la casse.

- b. Vérifiez que le **remote** est en cours d'exécution sur github.com.
- c. Affichez le **git log**. L'erreur indique qu'il n'y a pas de commits.

Étape 9: Stage et validation du fichier DEVASC.txt.

- a. Utilisez la commande **git add** pour mettre en scène le fichier DEVASC.txt.
- b. Utilisez la commande **git commit** pour valider le fichier DEVASC.txt.

Étape 10: Vérifiez la validation.

- a. Utilisez la commande **git log** pour vérifier la validation.
- b. Utilisez la commande **git status** pour afficher les informations d'état. L'expression "working tree clean" signifie que Git a comparé votre liste de fichiers à ce que vous avez dit à Git, et c'est une ardoise propre avec rien de nouveau à signaler.

Étape 11: Send (push) the file from Git to GitHub.

Utilisez la commande **git push origin master** pour envoyer (pousser) le fichier à votre dépôt GitHub. Vous serez invité à entrer un nom d'utilisateur et un mot de passe, qui sera celui que vous avez utilisé pour créer votre compte GitHub.

Remarque : Si, après avoir entré votre nom d'utilisateur et votre mot de passe, vous obtenez une erreur fatale indiquant que le dépôt est introuvable, vous avez probablement soumis une URL incorrecte. Vous devrez inverser votre commande **git add** avec la commande **git remote rm origin**.

Étape 12: Vérifiez le fichier sur GitHub.

- a. Accédez à votre compte GitHub et sous « Référentiels » sélectionnez le nom d' **utilisateur/devasc-study-team**.
- b. Vous devriez voir que le fichier DEVASC.txt a été ajouté à ce dépôt GitHub. Cliquez sur le fichier pour afficher le contenu.